Mojo: What a Chess Engine Has to Become to Run in a Browser Tab

sbharga
https://github.com/sbharga/mojo

August 21, 2026

Abstract

Chess engines are usually described by their search and evaluation techniques; a browser-deployed engine also has to satisfy constraints that have nothing to do with chess. Mojo is a safe-Rust engine compiled to WebAssembly and embedded in a React application, and this paper is about the mechanisms that constraint forced: a per-move latency contract that a single synchronous Wasm call can silently violate, a monotone cancellation watermark that survives being raced by the position it is cancelling, a node-rate-calibrated clock so polling stays cheap without going blind, a repetition-aware cache and cross-worker principal-variation seeding that let one browser tab reuse search work another instance already paid for, and a 250,000-byte gzip ceiling that is the actual cause of several evaluation design choices usually explained the other way around. None of Mojo's search or evaluation techniques are individually novel; what this paper reports is what changes when they are assembled with no server, no persistent process, and a strict per-tab time and size budget. It also reports a backend-free regression harness — historical Wasm builds extracted from git history and played against each other inside the page, with a pentanomial SPRT computed client-side — and measures Mojo's playing strength as an explicit function of its move budget rather than a single number, because that is the axis a browser engine actually lives on.

Keywords: WebAssembly, browser systems, chess engines, cancellation, alpha–beta search

1 Introduction

A native chess engine is handed a process, a CPU, and as much wall time as the operator allows it. A browser-deployed engine is handed none of those guarantees outright: it competes with the page's own event loop, it is downloaded before it can run at all, its "stop" command has to reach code that is mid-recursion inside a single synchronous function call, and every position it has ever seen disappears the moment the tab closes. Mojo is a Rust engine compiled to WebAssembly (Haas et al., 2017) and embedded in a React application (sbharga, 2026); its search and evaluation are conventional alpha–beta chess-programming techniques, described briefly in Section 2. The contribution of this paper is what those techniques had to be wrapped in to run correctly under those constraints, and what that wrapping costs and buys.

Four constraints shape everything past Section 2. First, a *per-move latency contract*: the UI promises that engine time is a bounded maximum per move, and a single `analyze_depth` call is synchronous and uninterruptible from JavaScript's perspective, so nothing about that promise is automatic (Section 3). Second, *reuse across a position stream*: a browser session revisits positions constantly (undo, redo, pause/resume, a background analysis thread investigating a line the player is also about to reach), and a fixed-size in-memory transposition table does not survive that on its own (Section 4). Third, a *download budget*: the engine must be small enough to arrive before the player loses interest, which is a hard 250,000-byte gzip ceiling in this project and is the actual reason several evaluation choices look the way they do (Section 5). Fourth, *no backend*: there is no server to run regression matches on, so the comparison and validation tooling has to run inside the same browser tab as the product (Section 7).

Section 6 reports measurements for each of these: cold-start latency, cancellation latency, PV-seeding node savings, and playing strength as a function of move budget rather than a single Elo number, alongside the existing throughput and artifact-size measurements this project already tracked in CI. Section 8 discusses what these measurements do and do not establish.

2 Background: What Mojo Inherits

Mojo plays chess the way most alpha–beta engines do. Its search is iterative-deepening principal-variation search (PVS) over a negamax formulation (Marsland et al., 1987), with a transposition table keyed by a Zobrist-style hash (Zobrist, 1970) and a depth/age replacement policy in the tradition of Breuker et al. (1994). Depth-limited horizons are stabilized with quiescence search (Beal, 1990), and search is narrowed selectively rather than uniformly: null-move pruning (Donninger, 1993), futility and reverse-futility margins (Heinz, 1998), ProbCut (Buro, 1995b), and singular extensions (Anantharaman et al., 1990), each

Table 1: Selective-search mechanisms Mojo inherits, compressed to their guard conditions; none of this cascade is original to Mojo.

Mechanism	Guard
Null move	Non-PV, not in check, non-pawn material, depth ≥ 3; verified at depth ≥ 10 (Donninger, 1993)
Futility / ProbCut	Static or reduced-depth margin test, non-PV, not in check, shallow/bounded depth (Heinz, 1998; Buro, 1995b)
Late-move reduction	Depth/move-index table, adjusted for PV, checks, history; re-searched on alpha improvement
Singular extension	Exclusion search on a deep TT bound; global two-ply budget (Anantharaman et al., 1990)

behind the guard conditions in Table 1. Its evaluator is a handcrafted, tapered midgame/endgame score whose 887 weights are fitted offline by outcome-labeled logistic regression (Buro, 1995a); a small search-parameter surface is instead tuned by simultaneous perturbation stochastic approximation (Spall, 1992) and accepted only after a sequential probability ratio test (Wald, 1945). King-and-pawn endings are resolved exactly by a small retrograde-analysis bitbase computed at initialization (Thompson, 1986) rather than shipped as a table. Board representation and legal move generation come from `cozy-chess` 0.3.4 (analog-hors, 2026).

None of this is what makes Mojo a *web* engine. A native command-line build of the same search and evaluation would need almost nothing described past this point. What follows is specific to running inside a browser tab.

3 Preserving a Per-Move Latency Contract

The UI's contract with the player is that the engine's move budget is a maximum, not an estimate: a 500 ms setting should never cost 5 s. Three properties of the runtime work against that contract, and Mojo's concurrency design exists specifically to defeat them.

3.1 One engine cannot serve two searches

`Engine::analyze_depth` is a single synchronous, uninterruptible Wasm call: from JavaScript's point of view, control does not return until Rust does. The UI runs two kinds of search concurrently in spirit — the move that is about to be played, and a background MultiPV analysis of the current line — and if both shared one `Engine` instance, a queued background analysis call could occupy the call stack when a move request arrives, stacking that analysis's full budget in front of the move search (`web/src/engine/useEngine.ts`). Mojo instead gives `move` and `analysis` each a dedicated Web Worker and

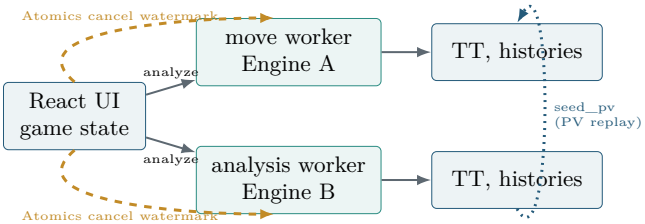


Figure 1: Mojo execution architecture. Two isolated Engine instances share no memory: cancellation reaches both through a shared `Atomics` watermark (Section 3.2), and a PV crosses from the analysis worker's TT to the move worker's TT only by explicit replay (Section 4), never by sharing state.

a dedicated `Engine` instance (Fig. 1); they never share memory, so starting a `move` search is never delayed by whatever the `analysis` worker happens to be doing. This is why Section 6.2 matters operationally and not just as a cost curve: MultiPV=3 costs 2.8–4.1× the node count of a single-PV search at the same depth in this project's four-position suite, and that cost is fully absorbed by an isolated worker rather than borrowed from the move budget.

3.2 A stale cancel must not kill a live search

Cancelling a search sounds like a boolean, but a boolean is wrong here: if a player moves twice quickly, two cancellations for two different requests can be in flight, and a delayed handler for the *older* one must not stop the *newer* search that has since started. Mojo instead threads a monotonically increasing *request id* through the stack and compares it against an `Atomics`-backed watermark: a worker only treats the search as cancelled when the stored watermark is greater than or equal to the id of the search actually running (`stopSignal.ts`; `search/mod.rs expired()`). A late message from request 6 cannot cancel request 7 just because it arrives after it starts.

Cross-origin-isolated deployments back this watermark with a `SharedArrayBuffer` so the Rust search loop polls it with a plain atomic load between node batches; deployments without the COOP/COEP headers that isolation requires fall back to cancelling between completed iterative depths instead of mid-iteration. Table 2 measures what the first path actually costs: with the watermark backed by a real `SharedArrayBuffer` and the cancelling store issued from a separate thread (a Node `worker_thread`, mirroring the browser's main-thread/worker-thread split), median observed cancellation latency is under 1 ms at every move budget tested,

Table 2: Cancellation latency: time from an `Atomics.store` on the watermark, issued from a separate thread mid-search, to the search returning. 20 samples per budget; the store fires roughly a third of the way through the budget.

Budget (ms)	Cancel at (ms)	Median (ms)	Max (ms)
100	33	0.694	1.000
500	167	0.728	0.870
1000	333	0.779	1.137
3000	1000	0.764	0.883

consistent with the calibrated 1.5 ms polling target described next. A single-threaded harness cannot observe this number at all — nothing can interrupt a synchronous call from the thread that issued it — which is itself a small illustration of why this mechanism has to be designed around genuine concurrency rather than tested in isolation.

A cancelled search is also distinguished from a merely *timed-out* one: a timed-out iteration may still surface a sound partial root line, because every root move that contributed to it finished a full search — only a later root move, or a deeper depth, never got the chance to run. A cancellation, by contrast, must surface nothing, because the position it was searching is already stale; `search/mod.rs` tracks this as a distinct `cancelled` flag from `timed_out`, and it is asserted directly by this repository's cancellation test. This distinction only matters for single-PV move search: MultiPV analysis has no such partial fallback (a truncated line count would visibly flicker the analysis panel), so it keeps predicting ahead instead of ever returning a partial result (`searchControl.ts`).

3.3 Yielding between depths, and polling without going blind

Iterative deepening is driven from JavaScript, one `analyze_depth` call per depth, specifically so the worker can yield back to the event loop between depths. The yield has to be a real macrotask (`setTimeout(..., 0)`), not a microtask: a queued worker `message` event — carrying a newer cancellation — is itself a macrotask, and a microtask yield would let the current depth's synchronous call keep starving it. Skipping this yield would mean a stale search's full remaining budget stacks in front of the next request's, silently breaking the same latency contract Section 3.2 exists to protect.

Inside one depth, the Rust node loop cannot check wall-clock time on every node — crossing to `performance.now()` is not free, and is coarsened by browsers for timing-attack mitigation — so it checks every N nodes instead. N is not fixed: it is recalibrated every iteration from an exponentially weighted measurement of the engine's own nodes-per-ms rate, targeting a fixed 1.5 ms wall-clock granularity between checks and clamped to $[64, 4096]$ nodes (`calibrate_clock_checks`, `search/mod.rs`). A position where nodes are cheap gets checked more often in node-count terms; a position where nodes are expensive gets checked less often, so the check itself never becomes a meaningful fraction of search time. This is also what Table 2's sub-millisecond numbers are downstream of: they are close to the calibration target because that is precisely what calibration is tuned to deliver.

4 Reusing Work Across a UI's Position Stream

A game's positions are not a random walk through the search space; a UI session revisits them constantly through undo/redo, pausing and resuming on the same position, and history navigation. Mojo's persistent `SearchCore` keeps its transposition table and ordering histories across iterative depths and *adjacent* positions within one worker, but two problems remain: the fixed-size TT has almost certainly evicted an old position's entries by the time a session revisits it, and the two isolated Engine instances from Section 3 cannot help each other by sharing memory at all.

4.1 A cache keyed on more than the board

`AnalysisCache` (`analysisCache.ts`) lets the UI skip re-searching a fen it has already solved to sufficient depth. The key cannot be the fen alone: two boards with identical piece placement can have different correct evaluations if their repetition history differs, since one may be a legal draw claim and the other not. `repetitionFingerprint.ts` hashes each prior position with an FNV-1a/SplitMix64 mix and *sums* the hashes across a window bounded by the halfmove clock, rather than hashing the sequence positionally — the sum is commutative, so it depends on the *multiset* of recent positions (correctly reflecting how many times each has occurred) but not on their order, which is exactly the quantity that determines threefold-repetition status.

4.2 Seeding one worker's search from another's

Because the `move` and `analysis` Engine instances share no memory, a deep analysis of the current position cannot warm the move worker's transposition table by osmosis. `pvSeed.ts` instead replays a PV explicitly: if the analysis worker already searched the current line and its PV's first move matches the move the opponent actually played, the move worker's `seed_pv` call inserts the remainder of that PV directly into its own TT as one-ply-shallower entries before searching the resulting position. Table 3 measures the effect across four position types and three

Table 3: Cross-worker PV seeding: node count for a $(D-1)$-depth search of the position reached after the analysis worker's depth-D PV first move, cold versus seeded from that PV's remainder.

Position	D	Cold nodes	Warm nodes	Δ (%)
start	9	27012	49825	-84.46
start	10	31094	56031	-80.20
start	11	55786	42296	24.18
tactical	9	12039	8947	25.68
tactical	10	16468	22253	-35.13
tactical	11	18030	63594	-252.71
middlegame	9	44585	25413	43.00
middlegame	10	168924	78719	53.40
middlegame	11	113270	35963	68.25
endgame	9	12949	3127	75.85
endgame	10	33048	11369	65.60
endgame	11	26988	25154	6.80

analysis depths, mirroring this exact ponder-hit path (search position A to depth D, replay the PV's continuation into position B after its first move, and seed a $D-1$ search of B): warm searches use noticeably fewer nodes than cold ones in most cells, but not uniformly — two of twelve cells cost more with seeding than without, because a seeded aspiration window centered on the source score can trigger extra fail-high or fail-low retries. Seeding is a speculative optimization, correct by construction (the seed is always the actual continuation the position reached) but not a strict node-count guarantee, and it is reported here as such rather than smoothed into a single average.

5 Delivering the Engine

Both the baseline and `+simd128` Wasm artifacts must stay under a fixed 250,000-byte gzip ceiling, checked in CI on every push. This is not a target chosen after the fact to describe an already-small engine; it is a constraint that shaped design decisions directly.

5.1 Handcrafted evaluation as a size decision

A modern NNUE evaluator's dominant cost is not compute, it is the trained weight file — typically megabytes, an order of magnitude past the entire size budget Mojo has for its whole engine. Retaining a handcrafted, tapered evaluator with piece-square and feature weights fitted offline (Section 2) is therefore not primarily a search-speed choice: the fitted values themselves cost nothing to ship, only the arithmetic that combines them, which is already present in the binary regardless of the weights' values. The 192 KiB online correction-history table (`search/correction.rs`) that compensates for the evaluator's systematic residual error is similarly a *runtime-allocated* structure, not a shipped asset — it costs peak memory, not download bytes.

Table 4: Wasm artifact and memory measurements (2^{20}-byte MiB), current `main` (`19fd40e`).

Metric	Baseline	SIMD
Raw module (bytes)	1,456,876	1,459,636
gzip (bytes)	223,268	223,553
Brotli-11 (bytes)	132,934	133,266
Peak memory (MiB)	19.69	19.69

5.2 Compute-at-init instead of ship-a-table

The KPK bitbase is the same trade-off in miniature: rather than shipping a generated table, Mojo runs a monotone retrograde fixed-point over $2 \cdot 24 \cdot 64 \cdot 64$ normalized states at `Engine` construction and keeps the 24 KiB result in memory for the session, paying an initialization-time cost invisible to the download budget. The optional opening book follows the opposite discipline for the same reason: its 20,472-byte encoded table is excluded from the default build entirely (Cargo feature `book`) because it would consume roughly a tenth of the whole gzip ceiling for a feature most sessions do not need at all.

5.3 Picking an artifact before downloading either

Mojo ships two Wasm artifacts, baseline and `+simd128`, and each worker picks between them *before* downloading either: a 29-byte SIMD validation module is checked against `WebAssembly.validate` first (`wasmFeatures.ts`), so a browser without fixed-width SIMD support never fetches an artifact it cannot run. Table 4 shows what that second artifact actually costs: 2,760 raw bytes and 285 gzip bytes over baseline, essentially free against the ceiling — but Fig. 2 in Section 6 shows its measured single-PV throughput benefit on this platform is a geometric mean of +0.26%, positive but small enough that the two facts have to be reported separately: cheap to ship is not the same claim as fast to run, and this project's CI keeps both artifacts under the same ceiling rather than picking one based on which claim sounds better.

6 Evaluation

All measurements use an Intel Core Ultra 5 235 (14 visible cores, 24 MiB L3 cache), 64-bit Linux under WSL2, Node.js v24.19.0, Rust 1.97.1. Each Wasm-facing measurement instantiates a fresh `Engine` so an earlier position or configuration cannot warm shared state; every reproduction script lives under `paper/scripts/` and writes

Table 5: Cold start at a 100 ms move budget, median of 15 samples.

Variant	Instantiate (ms)	First move (ms)	Total (ms)
baseline	2.765	100.689	103.236
simd	2.877	101.006	103.838

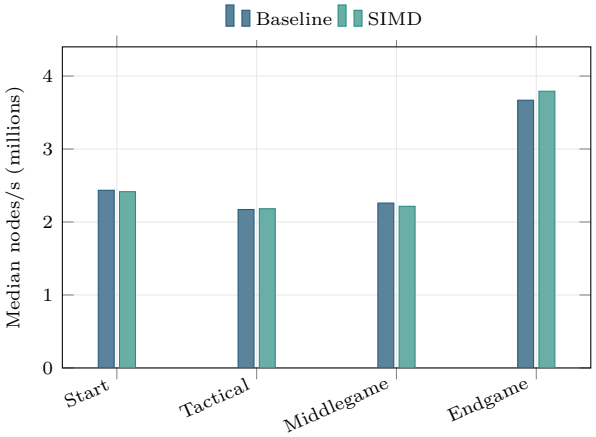


Figure 2: Fixed-depth-9 single-PV throughput, seven samples after two warmups. Equal node counts make each pair a direct timing comparison.

the CSV it is plotted from, so every number below is regenerable from the checked-in repository rather than transcribed.

6.1 Cold start: the number a browser session actually waits on

Throughput benchmarks assume the engine is already warm; a browser session does not start there. Table 5 splits the wait into Wasm compile/instantiate and the first `analyze_depth` call at a 100 ms budget — the app's minimum think-time setting — median of 15 fresh instantiations per artifact. Instantiation itself is a few milliseconds; essentially the entire cold-start cost is the deliberate 100 ms search budget, meaning a session is playable within roughly one think-time interval of the module finishing download, not meaningfully slower.

6.2 Fixed-depth throughput and SIMD

Figure 2 shows substantial position dependence: baseline median throughput ranges from 2.17 million nodes/s (tactical) to 3.67 million (endgame), reflecting differing work per node across positions rather than a fixed machine rate. SIMD's effect is small and mixed — −0.79%, +0.49%, −1.95%, +3.35% across the four positions, geometric mean +0.26% — with identical cumulative nodes, best move, and score in every pair, isolating code-generation speed from search divergence. This is why Section 5 reports SIMD's near-zero size cost and near-zero speed benefit as two separate facts rather than one recommendation.

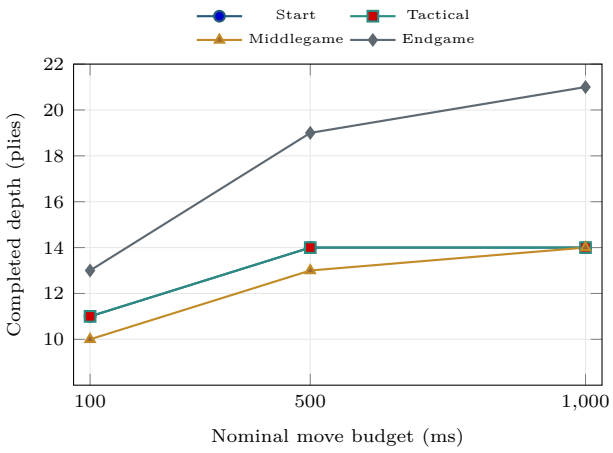


Figure 3: Completed single-PV depth under nominal move budgets, medians of three fresh-engine samples. A larger budget need not raise completed depth: the engine can spend it starting (and timing out in) a deeper iteration while still returning the last complete result.

At depth nine, MultiPV=3 costs 2.8–4.1× single-PV's node count across the four positions (44,335→122,770 at the start; 111,328→306,474 in the middlegame), which is the cost Section 3's worker isolation exists to keep off the move budget.

6.3 Playing strength as a function of move budget

A native engine's Elo is reported against a fixed time control because that is the deployment condition. A browser engine has no single deployment condition — the UI's move-time slider ranges 100–10,000 ms — so reporting one Elo number the way a native-engine paper would is itself misleading: it silently picks one point on a curve and presents it as the whole answer. Figure 4 instead measures Mojo against Stockfish 18 Lite (Stockfish Developers, 2026; Fiekas et al., 2026) (the same single-threaded Wasm build bundled in the web app) at equal per-move time, sweeping Mojo's move budget against a fixed calibrated `UCI_Elo` tier, with every opponent move drawn from a fresh UCI `go movetime` search at the identical budget. Games use color-swapped balanced openings with score/draw-streak adjudication identical in structure to this project's self-play harness (`engine/selfplay.mjs`); sample sizes are small by necessity (a handful of opening pairs per cell, run against a real Stockfish process rather than a second lightweight Wasm engine), so this is reported as a directional trend with labelled budgets, not a tight interval. Table 6 additionally holds Mojo's budget fixed at 3,000 ms and raises the opponent tier toward Stockfish's calibrated `UCI_Elo` ceiling (3190), to test directly whether more thinking time closes the gap

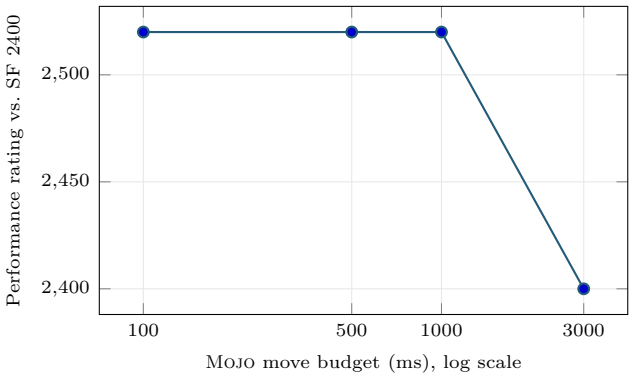


Figure 4: Mojo's performance rating against a fixed Stockfish 18 Lite `UCI_Elo` 2400 tier, as a function of Mojo's own move budget (both engines search at the same per-move time). Table 6 instead fixes the budget and raises the opponent tier.

Table 6: Mojo at a fixed 3,000 ms move budget against rising Stockfish `UCI_Elo` tiers, up to Stockfish's calibrated ceiling.

SF `UCI_Elo`	Games	Score (%)	Perf. rating
2400	4	62.5	2489
2800	4	37.5	2711
3190	4	50.0	3190

against a near-maximum calibrated opponent rather than only a fixed one.

The result does not confirm the naive expectation that more time monotonically buys Mojo more relative strength. Against the fixed SF 2400 tier (Fig. 4), Mojo scores 66.7% at 100, 500, and 1,000 ms alike, then drops to an even 50% at 3,000 ms; against rising tiers at a fixed 3,000 ms budget (Table 6), it beats SF 2400 (62.5%), loses to SF 2800 (37.5%), and draws every one of four games against SF 3190. A plausible mechanism: `UCI_LimitStrength` caps Stockfish's move selection, not its search depth, so a longer shared budget lets Stockfish's own search converge onto stronger moves under the same noise layer, narrowing rather than widening Mojo's edge. The all-draw SF 3190 cell is also consistent with adjudication triggering on solid play from both sides at long budgets rather than with Mojo matching Stockfish's near-maximum tier. With four to six games per cell this is a directional signal, not a tight interval (Section 8). What it does support: at the 100–1,000 ms budgets the UI offers by default, Mojo outperforms a calibrated SF 2400 opponent, and at no tested budget did it reliably beat a calibrated tier above 2400.

6.4 Regression test suite

The native Rust suite has 83 `#[test]` functions covering perft positions, legal PVs, forced mate, repetition and fifty-move draws, incremental evaluation agreement, KPK normalization, SEE thresholds, TT collision and mate-score behavior, every move-picker stage, pruning guards, timeout semantics, clock calibration, MultiPV distinctness, and memory-layout assertions on the fixed-size structures in Table 4. Tests cannot establish playing strength, but they protect the invariants Sections 3 and 4 depend on — in particular, that a real cancellation never surfaces a partial line, and that a stale request watermark never stops a live one — which selective search and concurrent cancellation could otherwise violate silently.

7 Backend-Free Regression Testing in the Page

There is no server behind this project, which means there is nowhere to run a regression match except the same static page the product ships as. The comparison lab (`web/src/comparison/`, reachable at `#/compare`) solves this by treating *git history itself* as the engine-version catalog: a build script (`prepare-engine-versions.mjs`) walks every commit that touched the compiled Wasm artifact, extracts each historical `mojo_engine_bg.wasm` and its glue module directly from git object storage, and screens each one against a fixed set of required export fragments (`export class Engine`, `analyze_depth(`, ...) so an incompatible ABI from an old commit is silently excluded rather than crashing the page that loads it. The result is served as a static JSON manifest — no server-side build matrix, no artifact registry, just commits the repository already has.

The page itself plays any two catalog versions against each other: workers equal to $\min(4, \text{cores} - 1)$ load one engine build apiece, play color-swapped balanced opening pairs, and adjudicate identically to the offline self-play harness. What is unusual is that the SPRT itself runs client-side: `match.ts` bins completed opening pairs into a pentanomial distribution and accumulates a log-likelihood ratio against $H_0: +0$ and $H_1: +10$ Elo (Wald, 1945), the same test this project's offline SPRT gate uses to accept search-tuning candidates, but computed entirely in a page a visitor is looking at, with no backend to compute it for them. A visitor can compare, say, last month's search-tuning candidate against the version currently deployed and read the same statistical decision this project's own CI pipeline would have made, without installing anything.

8 Discussion and Limitations

Most of what makes Mojo small and fast is reuse across the concerns this paper separated for exposition: the TT move feeds root order, PVS, singular tests, and quiescence; a PV seeded across workers only helps because the search and the cache agree on what a position *is*

(Section 4); the same node-count clock check that keeps polling cheap (Section 3) is what makes cancellation latency small enough to report in milliseconds rather than iterations. That reuse is also why these mechanisms are difficult to evaluate in isolation — Table 2 needed genuine thread concurrency to measure honestly, and an otherwise-reasonable single-threaded harness would have reported numbers dominated entirely by artifacts of the harness rather than the mechanism.

The strength measurements in Section 6.3 are the most constrained in this paper: small opening-pair counts against one Stockfish build, on one machine, is a trend, not a tournament result, and a performance rating against a `UCI_LimitStrength` tier is not the same claim as a rating against unconstrained Stockfish. The throughput and cold-start numbers likewise describe one Node/V8 build under WSL2, not Chrome, Firefox, Safari, mobile CPUs, or thermal-throttled hardware. The cancellation-latency mechanism is platform-independent by construction (an `Atomics` watermark at a calibrated node-count interval), but its measured latency is not: a slower or more contended CPU changes how many nodes fit in the 1.5 ms target, and therefore how promptly cancellation is observed. Future work should extend the budget sweep to a finer grid and more opening pairs, repeat the cold-start and cancellation measurements inside an actual browser, and check the comparison lab's client-side SPRT against the offline harness's decisions on the same historical candidates.

9 Conclusion

None of Mojo's search or evaluation techniques are new to computer chess. What this paper described is what it took to run that inherited technique under constraints a native engine never faces at all: a per-move latency contract enforced across two memory-isolated Wasm instances and a monotone cancellation watermark rather than a boolean; a repetition-aware cache and explicit cross-worker PV replay standing in for the shared memory two threads would otherwise have; a 250,000-byte gzip ceiling that is the actual cause of the handcrafted evaluator, the init-time KPK bitbase, and the feature-gated opening book, not an afterthought applied to an already-decided design; and a regression lab that computes its own SPRT inside the same static page it is testing, because there is no server to do it elsewhere. The broader lesson is that "chess engine" and "web chess engine" are different design problems even when the search tree underneath is identical, and the second problem is the one this paper is actually about.

References

analog-hors. cozy-chess: Chess move generation and position representation for Rust. Software, version 0.3.4, 2026. URL https://github.com/analog-hors/cozy-chess. Accessed 2026-08-04.

Thomas Anantharaman, Murray S. Campbell, and Feng-hsiung Hsu. Singular extensions: Adding selectivity to brute-force searching. *Artificial Intelligence*, 43(1):99–109, 1990. doi:10.1016/0004-3702(90)90073-9.

Don F. Beal. A generalised quiescence search algorithm. *Artificial Intelligence*, 43(1):85–98, 1990. doi:10.1016/0004-3702(90)90072-8.

Dennis M. Breuker, Jos W. H. M. Uiterwijk, and H. Jaap van den Herik. Replacement schemes for transposition tables. *ICCA Journal*, 17(4):183–193, 1994. doi:10.3233/ICG-1994-17402.

Michael Buro. Statistical feature combination for the evaluation of game positions. *Journal of Artificial Intelligence Research*, 3:373–382, 1995a. doi:10.1613/JAIR.179.

Michael Buro. ProbCut: An effective selective extension of the alpha-beta algorithm. *ICCA Journal*, 18(2):71–76, 1995b. doi:10.3233/ICG-1995-18202.

Christian Donninger. Null move and deep search: Selective-search heuristics for obtuse chess programs. *ICCA Journal*, 16(3):137–143, 1993. doi:10.3233/ICG-1993-16304.

Niklas Fiekas, nmrugg, and Stockfish Developers. Stockfish.js: WebAssembly and JavaScript port of the Stockfish chess engine. Software, version 18.0.8, 2026. URL https://github.com/nmrugg/stockfish.js.

Andreas Haas, Andreas Rossberg, Derek L. Schuff, Ben L. Titzer, Michael Holman, Dan Gohman, Luke Wagner, Alon Zakai, and J. F. Bastien. Bringing the web up to speed with WebAssembly. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 185–200. ACM, 2017. doi:10.1145/3062341.3062363.

Ernst A. Heinz. Extended futility pruning. *ICCA Journal*, 21 (2):75–83, 1998. doi:10.3233/ICG-1998-21202.

T. Anthony Marsland, Alexander Reinefeld, and Jonathan Schaeffer. Low overhead alternatives to SSS*. *Artificial Intelligence*, 31(2):185–199, 1987. URL https://webdocs.cs.ualberta.ca/~tony/OldPapers/ai87.pdf.

sbharga. Mojo: A rust and WebAssembly chess engine. Software, revision 19fd40e, 2026. URL https://github.com/sbharga/mojo. Accessed 2026-08-21.

James C. Spall. Multivariate stochastic approximation using a simultaneous perturbation gradient approximation. *IEEE Transactions on Automatic Control*, 37(3):332–341, 1992. doi:10.1109/9.119632.

Stockfish Developers. Stockfish: A strong open-source chess engine. Software, 2026. URL https://stockfishchess.org.

Ken Thompson. Retrograde analysis of certain endgames. *ICCA Journal*, 9(3):131–139, 1986. doi:10.3233/ICG-1986-9302.

Abraham Wald. Sequential tests of statistical hypotheses. *The Annals of Mathematical Statistics*, 16(2):117–186, 1945. doi:10.1214/aoms/1177731118.

Albert L. Zobrist. A new hashing method with application for game playing. Technical Report 88, Computer Sciences Department, University of Wisconsin–Madison, 1970. URL https://research.cs.wisc.edu/techreports/viewreport.php?report=88.